

Equipetrol Delivery

Guía de desarrollo paso a paso

Simulación concurrente de despacho en C++17. Plan completo por fases, arquitectura, interfaces, trampas, checkpoints y preguntas de defensa, alineado estrictamente con el brief del ingeniero.

Qué contiene	Para qué sirve
Secciones 1 a 5	Entender el problema, las reglas duras y la arquitectura completa antes de escribir código.
Sección 6 (Fases 0 a 11)	El trabajo en orden. Cada fase: objetivo, archivos, interfaz sugerida, pasos, trampas, checkpoint y preguntas de defensa.
Secciones 7 a 9	Validador del log, escenarios de prueba, herramientas (TSAN, ASan, Valgrind) y experimentos.
Secciones 10 y 11	Plantilla del README (esquema, mapa de conceptos, argumento de ausencia de races) y checklist final.
Apéndices	Prompt base para otra IA, equivalencias C++20 → C++17, problemas típicos en WSL.

Curso: Programación concurrente en C++ · UCB · Sexto semestre. Documento de tutoría: el código lo escribes tú.

0. Cómo usar este documento

Este documento es un **mapa**, no una solución. Da interfaces (firmas de clases y funciones), pseudocódigo de los bucles de cada hilo y los patrones exactos de sincronización que el curso enseñó. La implementación la escribes tú. Hay tres motivos para trabajar así:

1. El ingeniero **puede preguntarte oralmente cualquier línea** del mapa de conceptos y del esquema (sección 11 del brief, punto 9). Si no lo escribiste tú, no lo vas a poder defender.
2. El trabajo es **individual**.
3. Los defectos de concurrencia se entienden al escribirlos y romperlos, no al leerlos.

Ciclo de trabajo para cada fase

1. Lee la fase completa: objetivo, interfaz y **trampas**.
2. Escribe el código. Si consultas otra IA, pégale primero el **prompt base del Apéndice A**, para que no te proponga cosas prohibidas (C++20, **detach**, **sleep** para coordinar, etc.).
3. Compila con **-Wall -Wextra**: cero warnings.
4. Corre el **checkpoint** de la fase. Si falla, no avances.
5. Compila con **-DENABLE_TSAN=ON** y corre otra vez. TSAN tiene que salir limpio en **cada** fase, no solo al final.
6. Haz commit y anota en **NOTES.md** las decisiones de diseño que tomaste, porque son material para el README.
7. Responde las **preguntas de defensa** sin mirar el código.

Regla de oro al usar otra IA

Si la IA te sugiere algo de esta lista, recházalo: `std::jthread`, `std::stop_token`, `std::counting_semaphore`, `std::latch`, `std::barrier`, `std::atomic::wait`, `detach()`, `sleep_for` para esperar a otro hilo, `while(!flag) {}`, `m.lock()/m.unlock()` a mano, `new/delete`, OpenMP, TBB, Boost, `pthread_create`, `cv::parallel_for_`, `FetchContent` que descargue de internet.

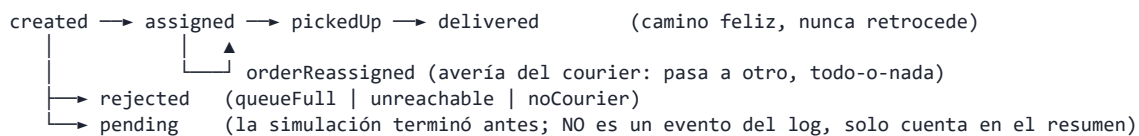
Si la IA escribe una clase completa, **pídele que te explique cada lock y cada espera** y reescríbela tú.

1. El proyecto en una página

Hay que construir `delivery_sim`, un programa **C++17** que simula en tiempo acelerado una empresa de delivery en **Zona Equipetrol** (Santa Cruz de la Sierra), entre el Segundo y el Cuarto Anillo. La simulación se dibuja en vivo sobre una imagen real de **OpenStreetMap**.

Actor	Qué hace	Recurso escaso que comparte
Generador de órdenes	Crea órdenes continuamente y en ráfagas (hasta <code>burstMax</code> a la vez).	Libro de órdenes (máx. <code>maxPending</code>)
Dispatcher	Elige el mejor courier por tiempo estimado de entrega. Las estimaciones se calculan en paralelo , con timeout.	Libro de órdenes, couriers
Couriers (motos)	Cada uno es independiente y simultáneo: van al restaurante, recogen, entregan y se averían.	Bolsa (<code>bagCapacity</code>), slots de restaurante
Restaurantes	Preparan comida y la entregan por <code>pickupSlots</code> ventanillas en orden de llegada (FIFO).	Slots de entrega
Ventana	Dibuja mapa, motos que se mueven fluido, entregas y contadores.	Lee snapshots de todos

Ciclo de vida de una orden



Invariante sagrado: cada orden termina en **exactamente un** estado final. Nunca se pierde, nunca se entrega dos veces y nunca está en dos bolsas a la vez. Se cumple `created = delivered + rejected + pending`.

Cómo se aprueba

- Pasar el **gate DP.0**: compila con `mkdir build && cd build && cmake .. && make`, cero warnings, opción `ENABLE_TSAN`, solo librerías permitidas.
- Obtener **≥ 70 puntos** de 100.
- Tener **puntaje completo en "No data races"** (8 pts): ThreadSanitizer no reporta **nada** en **ningún** escenario ni en **ninguna** corrida. Un solo race y se reprueba.

Criterio	Pts	Lo que más duele perder
DP.1 Ciudad real + config	6 + 4	Nodos que no caen sobre las calles; configs inválidas que no salen con código ≠ 0
DP.2 Órdenes	6	No reproducible con la seed; <code>maxPending</code> excedido
DP.3 Couriers concurrentes + idle	8 + 6	CPU > 0 sin trabajo; <code>sleep</code> usado como coordinación; motos cruzando manzanas
DP.4 Dispatch concurrente	8	Estimaciones secuenciales; la ventana se congela; una excepción tumba el programa
DP.5 Recursos escasos	8	Más couriers que slots; atención fuera del orden de llegada
DP.6 Incidentes atómicos	8	Orden en dos bolsas o en ninguna; deadlock con dos averías simultáneas
DP.7 Log veraz + sin races	8 + 8	Líneas mezcladas; contadores inexactos; cualquier race = reprobado
DP.8 Ventana	6	Puntos que saltan de intersección en intersección; ventana que se congela
DP.9 Parada limpia	6	Más de 5 s; exit ≠ 0; hilos abandonados
DP.10 / DP.11 / DP.12	6 + 6 + 6	Experimentos sin antes/después; salidas de herramientas faltantes; esquema que no coincide con el código

2. Reglas duras del ingeniero (checklist permanente)

Revisa esta tabla al final de cada fase. Cada fila indica de qué parte del brief sale la regla.

#	Regla	Qué significa en el código	Fuente
R1	C++17 estricto	Prohibido <code>jthread</code> , <code>stop_token</code> , <code>counting_semaphore</code> , <code>latch</code> , <code>barrier</code> , <code>atomic::wait</code> , corrutinas. Todo se construye con <code>mutex</code> + <code>condition_variable</code> .	§8, §10
R2	Solo stdlib + OpenCV + 1 JSON	OpenCV solo para la ventana (nada de <code>cv::parallel_for_</code> ; llama <code>cv::setNumThreads(0)</code>). JSON: <code>nlohmann/json.hpp</code> copiado en <code>third_party/</code> .	§8
R3	Build exacto	<code>mkdir build && cd build && cmake .. && make</code> y nada más. Sin flags extra obligatorios, sin descargas en el build.	§8
R4	Cero warnings	<code>-Wall -Wextra</code> . Includes de terceros como <code>SYSTEM</code> . <code>-Werror</code> solo como opción de desarrollo, nunca por defecto.	§8
R5	<code>ENABLE_TSAN</code>	<code>cmake -DENABLE_TSAN=ON ..</code> produce el mismo target <code>delivery_sim</code> con <code>-fsanitize=thread</code> .	§8
R6	Sin dormir para coordinar, sin spinning	<code>sleep/wait_until</code> solo para (a) tiempo simulado y (b) ritmo de frames. Esperar a otro hilo siempre es <code>cv.wait(1k, predicado)</code> .	§4 Time
R7	Idle = 0% CPU	Courier sin trabajo bloqueado en <code>cv.wait</code> . Nada de <code>while(true){ if(...) ; sleep(10ms); }</code> .	§4, §11.7
R8	Todo hilo con <code>join</code>	Nunca <code>detach()</code> . Todo <code>std::thread</code> se une en la secuencia de apagado.	§7, §10
R9	Solo locks RAIL	<code>lock_guard</code> , <code>unique_lock</code> , <code>scoped_lock</code> . Nunca <code>.lock()/unlock()</code> manual sobre un mutex.	§10 (06)
R10	Sin punteros crudos dueños	<code>shared_ptr/unique_ptr</code> . Nada de <code>new/delete</code> .	§10 (03)
R11	Log = resumen	Un JSON completo por línea; eventos y contadores se actualizan en la misma sección crítica.	§6
R12	Parada ≤ 5 s, exit 0	Toda espera debe poder despertarse con el stop.	§7
R13	Mapa OSM real	2do a 4to anillo + Av. San Martín visibles; "© OpenStreetMap contributors" en la ventana y en el README.	§3
R14	Grafo mínimo	≥ 25 nodos, ≥ 40 segmentos, ≥ 6 restaurantes repartidos. Respetar <code>oneWay</code> . Las motos van solo por segmentos.	§3
R15	stdout limpio	Por stdout solo sale <code>Simulation has started...</code> y, al final, la línea JSON del resumen. Todo lo de debug va a stderr.	§6
R16	Main thread = ventana	<code>imshow/waitKey</code> solo desde <code>main</code> .	Práctica OpenCV

3. Entorno de trabajo

No uses MinGW en Windows. ThreadSanitizer no funciona ahí, y sin TSAN no puedes demostrar que no hay races, que es obligatorio para aprobar. Ya tienes **WSL2 Ubuntu** con WSLg (**DISPLAY=:0**), así que las ventanas de OpenCV se abren directamente.

3.1 Instalación

```
sudo apt update
sudo apt install -y build-essential cmake pkg-config libopencv-dev valgrind clang gdb curl python3
# Versiones (anótalas para el README):
g++ --version | head -1 ; cmake --version | head -1 ; make --version | head -1
pkg-config --modversion opencv4 ; valgrind --version
```

3.2 Dónde tener el código

- **Opción rápida:** trabajar en `/mnt/c/Users/alfre/Documents/.../rider_and_threads`. Compila más lento (el disco de Windows desde WSL es lento), pero ves los archivos desde Windows.
- **Opción recomendada:** clonar el repo dentro de Linux (`~/rider_and_threads`) y abrirlo con VS Code usando la extensión **WSL** (`code .` desde la terminal de Ubuntu). Se compila mucho más rápido y TSAN corre mejor.
- Instala en VS Code la extensión **C/C++** o **clangd** dentro de WSL, para tener autocompletado con los headers de Linux.

3.3 Estructura final del repositorio

```
rider_and_threads/
├── CMakeLists.txt
├── README.md                    # informe final (DP.12)
├── config/
│   ├── equipetrol.json         # la config oficial (DP.1)
│   └── tests/                  # small, large, burst, slots1, bag1, breakdown1,
                                # unreachable, invalid_*.json
├── data/equipetrol.png         # imagen OSM
├── third_party/nlohmann/json.hpp
├── src/
│   ├── main.cpp                # arranque, render loop, señales, apagado
│   ├── Config.h / Config.cpp   # carga + validación del JSON
│   ├── CityGraph.h / .cpp      # grafo inmutable, Dijkstra, lat/lon -> px
│   ├── SimClock.h              # reloj simulado
│   ├── Order.h                 # tipo move-only
│   ├── EventLog.h / .cpp       # monitor: log + contadores del resumen
│   ├── OrderBook.h / .cpp      # monitor: cola acotada de órdenes sin asignar
│   ├── Restaurant.h / .cpp     # monitor: slots FIFO
│   ├── Courier.h / .cpp         # monitor + hilo: máquina de estados de la moto
│   ├── Fleet.h / .cpp          # contenedor de couriers + aviso de capacidad
│   ├── QuotePool.h / .cpp      # pool de hilos de cotización (promise/future)
│   ├── Dispatcher.h / .cpp     # hilo de despacho
│   ├── OrderGenerator.h / .cpp # hilo generador
│   └── Renderer.h / .cpp       # dibujo (solo en main)
├── tools/
│   ├── osm_extract.py          # (opcional) extrae nodos/calles de Overpass
│   ├── validate_log.py         # valida log + resumen contra todas las reglas
│   └── run_scenarios.sh        # corre cada escenario N veces con señales aleatorias
├── experiments/               # configs, logs y resúmenes crudos (DP.10)
└── analysis/                  # salidas completas sin editar de TSAN/ASan/Valgrind (DP.11)
```

4. Arquitectura

4.1 Hilos de ejecución

Cada hilo tiene un **destino deliberado**: se crea explícitamente en `main` y se une con `join()` en la secuencia de apagado.

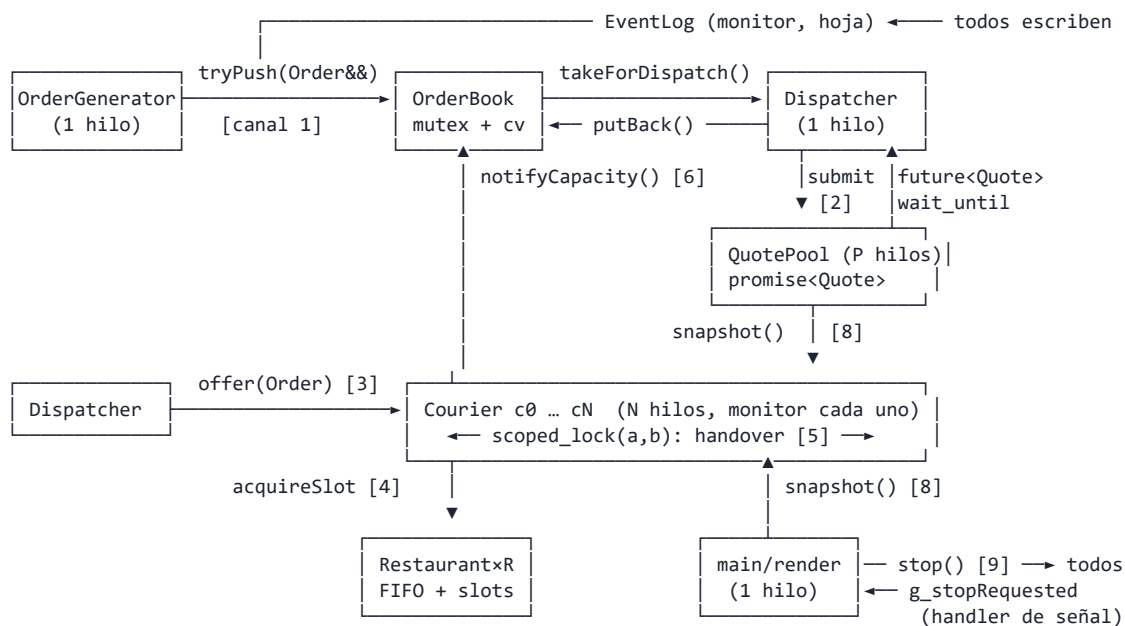
Hilo	Cuántos	Sobre qué hace loop	Cómo espera sin CPU
main / render	1	Dibuja un frame, revisa la señal y el fin de <code>durationS</code>	<code>cv::waitKey(33)</code> : ritmo de frames (permitido)
OrderGenerator	1	Espera un intervalo aleatorio (tiempo sim.) y crea de 1 a <code>burstMax</code> órdenes	<code>cv_.wait_until(realDeadline, pred stop)</code>
Dispatcher	1	Toma una orden del libro, pide cotizaciones y asigna	<code>OrderBook: wait</code> o <code>wait_until</code> (próximo vencimiento)
QuoteWorker	$P = f(\text{hardware_concurrency}())$	Toma una tarea de cotización, calcula el ETA y cumple la <code>promise</code>	<code>cv_.wait(pred hayTarea stop)</code>
Courier	<code>fleet.couriers</code>	Idle → a restaurante → recoger → a cliente → entregar (con avería posible)	Idle: <code>cv_.wait</code> ; viajando: <code>cv_.wait_until(finSegmento)</code> ; en restaurante: cv del restaurante

4.2 Objetos compartidos (monitores)

Un **monitor** es una clase con datos privados, un `std::mutex` privado y métodos públicos que toman ese mutex. No hay forma de tocar los datos sin el lock, porque nunca se devuelven referencias ni punteros a datos internos, solo copias o moves.

Objeto	Protege	Lo usan	Sincronización
<code>CityGraph</code>	Nodos, aristas, distancias precalculadas	Todos (solo lectura)	Immutable : se construye antes de crear hilos. No necesita lock.
<code>SimClock</code>	Instante de inicio, <code>timeScale</code>	Todos (solo lectura)	Immutable (<code>const</code>)
<code>EventLog</code>	Archivo, contadores, <code>t</code> , flag cerrado	Todos	mutex (hoja de la jerarquía)
<code>OrderBook</code>	Órdenes sin asignar, cupo, generación de capacidad	Generator, Dispatcher, Couriers	mutex + cv
<code>Restaurant</code>	Cola FIFO, slots en uso	Couriers	mutex + cv
<code>Courier</code>	Estado, bolsa, segmento actual, stop	Su hilo, Dispatcher, QuoteWorkers, Render, otro Courier (handover)	mutex + cv
<code>Fleet</code>	Contador de generación de capacidad	Couriers averiados	mutex + cv (la lista de couriers es immutable)
<code>QuotePool</code>	Cola de tareas	Dispatcher, QuoteWorkers	mutex + cv
<code>g_stopRequested</code>	Flag de señal	Handler de señal, main	<code>std::atomic<bool></code> lock-free

4.3 Esquema de hilos y canales (este diagrama va al README)



#	Canal	Tipo	Quién espera a quién
1	Generator → OrderBook → Dispatcher	Productor/consumidor acotado (monitor + cv)	Dispatcher espera órdenes
2	Dispatcher → QuotePool → Dispatcher	Cola de tareas + promise/future , wait_until	Workers esperan tareas; Dispatcher espera cotizaciones (máx. quoteTimeoutMs)
3	Dispatcher → Courier	Llamada a monitor offer() + notify al cv del courier	Courier idle espera trabajo
4	Courier ↔ Restaurant	Monitor con cola FIFO + cv	Courier espera turno y slot libre
5	Courier averiado → Courier receptor	Dos locks a la vez (std::scoped_lock)	Nadie espera; operación atómica
6	Courier → OrderBook / Fleet	notifyCapacity() (contador de generación + notify_all)	Dispatcher y couriers averiados esperan capacidad
7	Todos → EventLog	Monitor	Exclusión mutua breve
8	Render / Workers ← Courier	snapshot() devuelve una copia	Exclusión mutua breve
9	main → todos	stop() en cada monitor (flag + notify_all)	Despierta a todos para apagar

4.4 Jerarquía de locks (así se evita el deadlock)

Un hilo **solo** puede tomar un lock de un nivel **mayor** mientras sostiene uno de nivel menor. Nunca al revés. La única excepción es el handover, que toma **dos locks del mismo nivel a la vez** con **std::scoped_lock**, que aplica un algoritmo que evita el deadlock.

Nivel	Mutex	Puede tomar después	Nunca
1	Courier::m_ (uno, o dos con scoped_lock)	EventLog, Fleet	OrderBook, Restaurant, QuotePool
2	OrderBook::m_ , Restaurant::m_ , QuotePool::m_ , Fleet::m_	EventLog	Otro del nivel 2 ni ningún Courier
3	EventLog::m_	nada (hoja)	cualquier otro

Consecuencias prácticas de la jerarquía

El Dispatcher **suelta** el lock del OrderBook antes de llamar a **courier->offer()**: **takeForDispatch()** devuelve la orden por move y retorna.

El courier **no sostiene su propio lock** mientras espera en la cola del restaurante. Si lo hiciera, el render, los quotes y el handover se bloquearían detrás de él.

El courier llama a `book->notifyCapacity()` **después** de soltar su propio lock.

Nunca se espera en un cv sosteniendo un mutex **distinto** al del propio cv.

4.5 Máquina de estados del courier

Estado	Color	Pasa a	Cuándo
Idle	gris (quieto)	ToRestaurant / Delivering	<code>offer()</code> le da trabajo, o todavía lleva órdenes
ToRestaurant	azul (moviéndose)	AtRestaurant / Broken	llega al nodo del restaurante / avería en el camino
AtRestaurant	azul (quieto en la cola)	Delivering	obtiene un slot FIFO, espera la comida y registra <code>orderPickedUp</code>
Delivering	verde (moviéndose)	Delivering / Idle / Broken	entrega y sigue con la siguiente / bolsa vacía / avería en el camino
Broken	rojo (quieto)	Idle	handover todo-o-nada a otro courier y luego reparación (<code>kRepairMs</code>)

5. Las 9 ideas de diseño que hay que entender antes de programar

5.1 Movimiento fluido con 0% CPU: el courier duerme y el render interpola

El courier **no** se despierta en cada frame. Al entrar a un segmento publica, bajo su lock, "voy del nodo A al nodo B, empecé en `t0` y termino en `t1`" (tiempo simulado). Después duerme con `cv_.wait_until(realDeadline(t1), [&]{ return stop_; })`. El render, en cada frame, lee ese snapshot y calcula $\text{frac} = (\text{simNow} - t0) / (t1 - t0)$ y $\text{pos} = A + \text{frac} \cdot (B - A)$. Así el punto avanza suave, el courier no gasta CPU y un Ctrl-C lo despierta al instante.

5.2 Tiempo simulado

`simNow = (steady_clock::now() - start) × timeScale`. Toda espera **de simulación** (viajar, cocinar, entregar, intervalo entre órdenes, reparación) se convierte a un `time_point` real y se hace con `wait_until con predicado de stop`. Usa siempre `steady_clock`, nunca `system_clock`: el reloj del sistema puede saltar si alguien cambia la hora.

```
// SimClock.h (utilidad pequeña; puedes usarla tal cual)
#pragma once
#include <chrono>
#include <cstdint>

class SimClock {
public:
    using Real = std::chrono::steady_clock;
    explicit SimClock(double timeScale) : start_(Real::now()), scale_(timeScale) {}

    std::int64_t nowMs() const { // ms simulados desde el inicio
        std::chrono::duration<double, std::milli> real = Real::now() - start_;
        return static_cast<std::int64_t>(real.count() * scale_);
    }
    Real::time_point toReal(std::int64_t simMs) const { // instante real en que sim == simMs
        std::chrono::duration<double, std::milli> real(static_cast<double>(simMs) / scale_);
        return start_ + std::chrono::duration_cast<Real::duration>(real);
    }
private:
    const Real::time_point start_;
    const double scale_;
};
```

5.3 El patrón de espera correcto (se usa en todos los monitores)

```
// Espera "de simulación" interrumpible. Devuelve true si el tiempo pasó normalmente,
// false si nos despertaron para apagar.
bool Courier::sleepUntilSim(std::unique_lock<std::mutex>& lk, std::int64_t simDeadline) {
    bool stopped = cv_.wait_until(lk, clock_>toReal(simDeadline), [this] { return stop_; });
    return !stopped; // wait_until con predicado devuelve el valor del predicado
}

// Espera "por otro hilo": SIN timeout, SIN sleep, con predicado (robusto a despertares espurios).
cv_.wait(lk, [this] { return stop_ || !assigned_.empty() || !carried_.empty(); });

// stop() de cualquier monitor: flag bajo lock + notify_all
void Courier::stop() {
    { std::lock_guard<std::mutex> lk(m_); stop_ = true; }
    cv_.notify_all();
}
```

5.4 Por qué las cotizaciones NO usan std::async

El `std::future` que devuelve `std::async(std::launch::async, ...)` **bloquea en su destructor** hasta que la tarea termina. Si una cotización llega tarde y el dispatcher descarta su future, el dispatcher se queda congelado esperándola, justo lo que el brief prohíbe. Por eso las cotizaciones van a un **pool propio**: cada tarea lleva una `std::promise<Quote>`. Si llega tarde, llena un future que nadie lee y nadie se bloquea. `std::async(std::launch::async, ...)` se usa donde **sí** queremos esperar el resultado: el **precálculo de caminos mínimos** al arrancar, con una tarea por restaurante en paralelo.

5.5 Log igual a resumen, por construcción

`EventLog::emit()` toma el lock, calcula `t = clock.nowMs()` **dentro** del lock, escribe la línea completa terminada en `\n` y actualiza los contadores. Como todo ocurre en la misma sección crítica, el log y el resumen no pueden discrepar, `t` es monótono en el archivo y no se pueden mezclar dos líneas. Al registrar `simulationStopping` el log se **cierra**: los eventos posteriores se descartan, y esas órdenes quedan como `pending`, lo cual es coherente.

5.6 FIFO con slots sin semáforos (C++17)

`Restaurant` tiene `std::deque<std::string> queue_` (ids de courier en orden de llegada) e `int inUse_`. Un courier se encola al llegar y espera `cv_.wait(1k, [&]{ return stop_ || (queue_.front()==me && inUse_ < slots_); })`. Cuando le toca: `pop_front`, `++inUse_`, registra `pickupStarted` **dentro del lock** (así el orden del log es el orden real) y hace `notify_all` para que el siguiente revise si también cabe. Al liberar: `--inUse_` y `notify_all`.

5.7 Órdenes que existen en un solo lugar (move-only)

`Order` tiene el constructor de copia **borrado** y el de move por defecto. Viaja por move: Generator → OrderBook → Dispatcher → Courier (`assigned_`) → Courier (`carried_`) → (handover) otro Courier → destruida al entregarse. El compilador **impide** duplicarla. Para dibujar se usan copias de datos ligeros (ids y nodos), nunca la `Order`.

5.8 Señales: solo levantar un flag

```
// main.cpp
static std::atomic<bool> g_stopRequested{false};
static_assert(std::atomic<bool>::is_always_lock_free, "necesario para usarlo en un handler");
extern "C" void onSignal(int) { g_stopRequested.store(true); } // lo único seguro en un handler
// ... std::signal(SIGINT, onSignal); std::signal(SIGTERM, onSignal); ANTES de crear hilos
```

En un proceso multihilo la señal puede entregarse a **cualquier** hilo. Un `volatile sig_atomic_t` leído desde otro hilo es formalmente un data race. En cambio, un `std::atomic<bool>` lock-free es seguro en el handler (C++17 [support.signal]) y entre hilos. El render loop ya corre a ~30 fps, así que revisa el flag en cada frame y eso no es spinning.

5.9 Parada en orden y en menos de 5 s

```
1. generator->stop(); genThread.join(); // deja de crear órdenes
2. log->stopping(); // escribe simulationStopping y CIERRA el log
3. book->stop(); for (r : restaurants) r->stop(); for (c : couriers) c->stop();
   fleet->stop(); pool->stop(); // despierta a TODOS (flag + notify_all)
4. dispatcherThread.join(); for (t : courierThreads) t.join(); pool->join();
5. Summary s = log->summary(); std::cout << toJson(s) << std::endl; cv::destroyAllWindows();
6. return 0;
```

Cada espera tiene predicado de stop, así que todos despiertan de inmediato. El dispatcher puede tardar como máximo `quoteTimeoutMs` (unos 200 ms reales) si estaba esperando cotizaciones.

6. Fases de desarrollo

El orden está pensado para que **siempre** tengas algo que compila y corre, y para construir el **apagado temprano (Fase 3)**. Añadir el apagado al final obliga a reescribir todos los bucles.

Fase	Contenido	Rúbrica	Tamaño
0	Entorno + CMake + ventana vacía	DP.0	S
1	Mapa OSM + grafo + config + validación	DP.1	L
2	Piezas base: grafo, reloj, Order, EventLog	DP.7 (base)	M
3	Couriers que se mueven + render + apagado	DP.3, DP.8, DP.9	L
4	Generador + OrderBook	DP.2	M
5	Dispatcher simple + restaurantes + ciclo completo	DP.5, DP.3	L
6	Cotizaciones concurrentes (pool + promise/future)	DP.4	M
7	Averías + handover atómico	DP.6	M
8	Validador + escenarios	DP.7	M
9	TSAN / ASan / Valgrind	DP.11, No data races	M
10	Experimentos	DP.10	M
11	README final	DP.12	M

Fase 0: Entorno, CMake y ventana vacía

Objetivo	Rúbrica	Conceptos del curso
Tener un proyecto que compila con los comandos exactos, sin warnings, con opción TSAN, y que abre una ventana.	DP.0 (gate): sin esto no te revisan nada.	Ninguno aún. Es infraestructura.

Pasos

1. Instala el toolchain (sección 3.1).
2. Crea las carpetas y descarga el JSON: `mkdir -p src config/tests data third_party/nlohmann tools experiments analysis curl -L -o third_party/nlohmann/json.hpp https://github.com/nlohmann/json/releases/download/v3.11.3/json.hpp`
3. Escribe `CMakeLists.txt` con los requisitos de abajo.
4. Escribe un `src/main.cpp` que abra una ventana negra de 800×600 y salga con ESC.
5. Crea un `.gitignore` con `build*/`, `*.log`.

Requisitos del `CMakeLists.txt`

- `cmake_minimum_required(VERSION 3.11)` y `project(delivery_sim CXX)`.
- `CMAKE_CXX_STANDARD 17`, `CMAKE_CXX_STANDARD_REQUIRED ON`, `CMAKE_CXX_EXTENSIONS OFF`.
- Si no hay `CMAKE_BUILD_TYPE`, usa `RelWithDebInfo` por defecto (optimiza y deja símbolos para las herramientas).
- `find_package(OpenCV 4.1 REQUIRED)`, `find_package(Threads REQUIRED)`. El enlace con `Threads::Threads` es lo que necesita `std::thread`, **no** es usar hilos de plataforma directamente.
- `file(GLOB SOURCES ${PROJECT_SOURCE_DIR}/src/*.cpp)` y `add_executable(delivery_sim ${SOURCES})`.
- `target_include_directories(delivery_sim SYSTEM PRIVATE ${PROJECT_SOURCE_DIR}/third_party ${OpenCV_INCLUDE_DIRS})`.
- `target_compile_options(delivery_sim PRIVATE -Wall -Wextra)`.

- `option(ENABLE_TSAN "... OFF)` y `option(ENABLE_ASAN "... OFF)`. Si ambas están en ON, `message(FATAL_ERROR ...)`.
- Con TSAN: compilar con `-fsanitize=thread -g -O1 -fno-omit-frame-pointer` y enlazar con `target_link_libraries(delivery_sim PRIVATE -fsanitize=thread)` (`target_link_options` requiere CMake 3.13 y el mínimo es 3.11).
- Con ASAN: lo mismo con `-fsanitize=address,undefined`.

Trampas

-**Werror** activado por defecto: si el compilador del revisor genera un warning nuevo, **no compila y pierdes el gate**. Ofrecélo como opción `DEV_WERROR` desactivada.

`FetchContent` o `ExternalProject` que descargan cosas: en una máquina sin red falla el build.

Olvidar que la ventana tiene que existir siempre: "no hay modo sin ventana".

Checkpoint Fase 0

`mkdir build && cd build && cmake .. && make: cero warnings; ./delivery_sim` abre una ventana.

`mkdir build-tsan && cd build-tsan && cmake -DENABLE_TSAN=ON .. && make: compila. nm -C delivery_sim | grep -c __tsan > 0.`

`grep -n 'Werror' CMakeLists.txt` no muestra que esté activado por defecto.

Fase 1: Mapa real de OpenStreetMap, grafo y configuración

Objetivo	Rúbrica	Conceptos del curso
Imagen OSM de Equipetrol (2do a 4to anillo), bounds exactos, grafo real (≥ 25 nodos, ≥ 40 segmentos, ≥ 6 restaurantes), <code>config/equipetrol.json</code> , carga y validación.	DP.1 (6 + 4 pts)	Ninguno directo. Prepara el grafo immutable , que será el objeto compartido más fácil de justificar.

1.A Obtener la imagen con bounds exactos

1. Ve a openstreetmap.org y centra el mapa en Equipetrol. Av. San Martín cruza del 2do al 4to anillo; ahí está la zona.
2. Pestaña **Export** → "Manually select a different area": ajusta la caja para que contenga el 2do, 3er y 4to anillo y Av. San Martín completa. Anota los valores **N, S, W, E** que aparecen.
3. Botón **Share** (icono a la derecha) → **Image** → formato PNG → marca "Set custom dimensions" usando la misma área → Download. Guárdala en `data/equipetrol.png`.
4. Alternativa exacta: la URL de exportación tiene la forma `https://render.openstreetmap.org/cgi-bin/export?bbox=W,S,E,N&scale=...&format=png`. Si la construyes tú, los bounds son **exactamente** los que escribiste. Si el servidor está saturado, reintenta más tarde.
5. Verifica en la imagen que se vean y reconozcan el Segundo, Tercer y Cuarto Anillo y Av. San Martín, porque es requisito explícito.

Caja aproximada para empezar (**verificala en el mapa, no la copies a ciegas**): $N \approx -17.750$, $S \approx -17.782$, $W \approx -63.212$, $E \approx -63.180$.

1.B Conversión lat/lon → píxel

```
// Proyección lineal con los bounds. En ~3 km el error frente a Mercator es menor que un píxel.
cv::Point2f toPixel(double lat, double lon) const {
    float x = static_cast<float>((lon - west_) / (east_ - west_) * imgWidth_);
    float y = static_cast<float>((north_ - lat) / (north_ - south_) * imgHeight_); // y crece hacia abajo
    return {x, y};
}
```

1.C Obtener nodos y calles reales

Opción recomendada: Overpass Turbo (overpass-turbo.eu, datos OSM, licencia ODbL). Consulta, con tu `bbox` en el orden (S,W,N,E):

```
[out:json][timeout:60];
way["highway"~"^(primary|secondary|tertiary|residential|unclassified)$"](-17.782,-63.212,-17.750,-63.180);
out geom;
```

Exporta el resultado como JSON y escribe `tools/osm_extract.py`, o pídele a otra IA que te ayude con este script, porque no es parte concurrente. Pasos del script:

1. Cuenta en cuántas **ways** aparece cada nodo OSM. Los que aparecen en ≥ 2 **ways**, o son extremos de una way, son **intersecciones**.
2. Recorre cada way y parte en segmentos entre intersecciones consecutivas.
3. **Muy importante:** los anillos son **curvos**. Si unes dos intersecciones lejanas con una línea recta, la moto cruza manzanas, y eso es un defecto explícito. Conserva nodos intermedios de forma (grado 2) cada vez que la curva se desvíe más de unos 8 px de la recta. Esos nodos también van a **nodes**.
4. **oneWay:** `true` si la way tiene `oneway=yes`. Si tiene `oneway=-1`, invierte `from/to`.
5. Filtra para quedarte con unos **40 a 120 nodos** repartidos por toda la zona. Mínimo: 25 nodos y 40 segmentos.
6. Elige ≥ 6 **restaurantes** repartidos: por ejemplo 2 cerca del 2do anillo, 2 cerca del 3ro y 2 cerca del 4to. Asigna cada uno a su nodo más cercano. Puedes buscar `amenity=restaurant` en OSM para usar nombres reales.
7. Genera `config/equipetrol.json` con el formato exacto de la §5 del brief.

Opción manual (si Overpass te complica): haz clic en openstreetmap.org, en cada intersección copia la lat/lon de la URL y construye el JSON a mano. Es lento, pero para unos 40 nodos es viable.

1.D Carga y validación de la configuración

```
// Config.h (interfaz sugerida)
struct Bounds { double north, south, west, east; };
struct NodeCfg { std::string id; double lat, lon; };
struct StreetCfg { std::string id, from, to; bool oneWay; };
struct RestaurantCfg { std::string id, name, node; int pickupSlots; std::int64_t prepMin, prepMax; };
struct Config {
    std::filesystem::path imagePath; // YA resuelto relativo al archivo de config
    std::string attribution; Bounds bounds;
    std::vector<NodeCfg> nodes; std::vector<StreetCfg> streets; std::vector<RestaurantCfg> restaurants;
    int couriers, bagCapacity; double speedKmh; std::string startNode;
    std::int64_t meanIntervalMs; int burstMax; int maxPending; std::uint32_t seed;
    std::int64_t quoteTimeoutMs, acceptTimeoutMs;
    double breakdownProbability; std::int64_t durationS; double timeScale;
};
class ConfigError : public std::runtime_error { using std::runtime_error::runtime_error; };
Config loadConfig(const std::string& path); // lanza ConfigError con un mensaje legible
```

Casos que deben terminar en **mensaje a stderr + exit $\neq 0$ sin simular**:

Caso	Cómo detectarlo
Falta el argumento	<code>argc < 2</code> → imprime el uso y <code>return 2</code>
No se puede leer el archivo	<code>std::ifstream</code> falla al abrir
JSON malformado	atrapar <code>nlohmann::json::parse_error</code>
Falta una propiedad requerida	usar <code>j.at("fleet").at("couriers")</code> , que lanza <code>out_of_range</code> ; atrápalo y di qué propiedad falta
Tipo incorrecto (<code>"couriers": "8"</code>)	<code>get<int>()</code> lanza <code>type_error</code>
Calle o restaurante con nodo inexistente	construye un <code>unordered_map<string,int></code> de ids y verifica
<code>startNode</code> inexistente, ids de nodo duplicados	misma verificación
Rangos: <code>couriers ≥ 1</code> , <code>bag ≥ 1</code> , <code>slots ≥ 1</code> , <code>burstMax ≥ 1</code> , <code>0 $\leq p \leq 1$</code> , <code>timeScale > 0</code> , <code>prep min $\leq$ max</code> , <code>durationS ≥ 0</code> , <code>north > south</code> , <code>east > west</code>	validaciones explícitas
La imagen no se puede cargar	<code>cv::imread(...).empty()</code> : valida antes de arrancar hilos
Flag desconocido o <code>--log</code> sin ruta	parseo de <code>argv</code>

La ruta de `map.image` es **relativa al archivo de configuración**:
`std::filesystem::path(cfgPath).parent_path() / image`.

Checkpoint Fase 1

El programa dibuja **todos los nodos y segmentos** sobre la imagen y **caen exactamente sobre las calles**. Toma una captura: te va a servir para el README.

Hay ≥ 25 nodos, ≥ 40 segmentos y ≥ 6 restaurantes, repartidos entre los anillos.

Cada config inválida de `config/tests/invalid_*.json` sale con código $\neq 0$ y un mensaje claro (`echo $?`).

La atribución "© OpenStreetMap contributors" se ve en la ventana.

Preguntas de defensa

- ¿Por qué tu proyección lineal es aceptable si OSM usa Mercator?
- ¿Cómo garantizas que una moto nunca cruza una manzana?

Fase 2: Piezas base (todavía sin hilos)

Objetivo	Rúbrica	Conceptos del curso
Grafo con Dijkstra y <code>oneWay</code> , reloj simulado, <code>Order</code> move-only, <code>EventLog</code> monitor y precálculo concurrente de distancias.	Base de DP.2, DP.4 y DP.7	02 (move), 05 (tipo seguro de mover), 04 (<code>std::async</code> con launch policy), 07 (monitor)

2.A CityGraph (immutable)

```
class CityGraph {
public:
    explicit CityGraph(const Config& cfg);           // ids string -> índices int
    int nodeIndex(const std::string& id) const;
    std::size_t nodeCount() const;
    double segmentMeters(int from, int to) const;   // haversine
    // Ruta más corta respetando oneWay. Vacía si no hay camino.
    std::vector<int> shortestPath(int from, int to) const;
    std::vector<double> distancesFrom(int src) const; // Dijkstra completo
    cv::Point2f pixelOf(int node) const;
private:
    struct Edge { int to; double meters; };
    std::vector<std::vector<Edge>> adj_; // si oneWay==false, se agregan ambas direcciones
    std::vector<double> lat_, lon_;
    /* ... */
};
```

Justificación para el README: se construye completo en `main` antes de crear cualquier hilo y después nadie lo modifica (todos sus métodos son `const`). La creación de un `std::thread` *sincroniza-con* el inicio de su función, así que todo lo escrito antes es visible y no hay race posible. Se comparte como `std::shared_ptr<const CityGraph>`.

2.B Precálculo concurrente con `std::async` (concepto 04a)

```
// Una tarea por restaurante, lanzada explícitamente en otro hilo:
std::vector<std::future<std::vector<double>>> futs;
for (int rNode : restaurantNodes)
    futs.push_back(std::async(std::launch::async,
                             [graph, rNode] { return graph->distancesFrom(rNode); }));
for (auto& f : futs) distFromRestaurant.push_back(f.get()); // get() también propaga excepciones
```

Para qué sirve: detectar `unreachable` en $O(1)$ (`dist[r][delivery] == inf`) y dar una base a las cotizaciones. Por qué `launch::async`: la política por defecto puede ser `deferred` y entonces todo correría secuencialmente dentro de `get()`. Aquí queremos esperar el resultado, así que el destructor bloqueante del future no molesta.

2.C Order (move-only)

```

struct Order {
    std::string id;           // "o17"
    std::string restaurantId;
    int restaurantNode = -1;
    int deliveryNode = -1;
    std::int64_t createdMs = 0;    // tiempo simulado
    std::int64_t readyAtMs = 0;    // createdMs + prepTime aleatorio

    Order() = default;
    Order(const Order&) = delete;    // nunca dos copias de la misma orden
    Order& operator=(const Order&) = delete;
    Order(Order&&) noexcept = default;    // viaja por move
    Order& operator=(Order&&) noexcept = default;
    ~Order() = default;
};

```

2.D EventLog (monitor, hoja de la jerarquía)

```

struct Summary { long created=0, delivered=0, rejected=0, pending=0, reassigned=0, breakdowns=0;
                  std::int64_t sumDeliveryMs=0, maxDeliveryMs=0;
                  std::map<std::string,long> deliveredPerCourier; };    // map: salida ordenada

class EventLog {
public:
    EventLog(const std::string& path, std::shared_ptr<const SimClock> clock,
              const std::vector<std::string>& courierIds);    // inicializa deliveredPerCourier en 0
    void orderCreated(const std::string& order, const std::string& restaurant);
    void orderAssigned(const std::string& order, const std::string& courier);
    void pickupStarted(const std::string& order, const std::string& courier, const std::string& rest);
    void orderPickedUp(const std::string& order, const std::string& courier, const std::string& rest);
    void orderDelivered(const std::string& order, const std::string& courier, std::int64_t createdMs);
    void orderRejected(const std::string& order, const std::string& reason);
    void courierBreakdown(const std::string& courier);
    void orderReassigned(const std::string& order, const std::string& from, const std::string& to);
    void stopping();    // escribe simulationStopping y cierra
    Summary summary() const;    // copia; pending = created - delivered - rejected
private:
    void writeLocked(nlohmann::json& ev);    // PRE: lock tomado. Pone "t", dump(), '\n'
    mutable std::mutex m_;
    std::ofstream out_;
    std::shared_ptr<const SimClock> clock_;
    Summary s_;
    bool closed_ = false;
};

```

Trampas de la Fase 2

Calcular **t fuera** del lock: dos hilos pueden escribir en el orden inverso a sus **t**, y una orden puede tener **t** decreciente.

Usar **std::endl** en cada línea del log: fuerza un flush por evento y es lento con muchas órdenes. Usa **'\n'** y haz **flush()** en **stopping()** y en el destructor.

nlohmann::json ordena las claves alfabéticamente por defecto. Eso es válido (JSON no exige orden), pero si prefieres **t** primero usa **nlohmann::ordered_json**.

Olvidar incluir en **deliveredPerCourier** a los couriers con 0 entregas.

avgDeliveryMs: define y documentas si es **delivered_t - created_t** (recomendado) y usa división entera si hay 0 entregas.

Checkpoint Fase 2

Un test pequeño (o modo debug en stderr) imprime una ruta entre dos nodos, y una ruta hacia un nodo aislado da vacío.

Escribir 1000 eventos desde 8 hilos a la vez produce 1000 líneas válidas (**python3 -c con json.loads** por línea). TSAN limpio.

Intentar copiar una **Order** **no compila**.

Preguntas de defensa

- ¿Por qué **CityGraph** no necesita mutex?
- ¿Qué pasaría si **std::async** usara la política por defecto?
- ¿Por qué el move constructor de **Order** es **noexcept**? (Pista: **std::vector** y **deque** al realojar.)

Fase 3: Couriers que se mueven, render y apagado limpio

Objetivo	Rúbrica	Conceptos del curso
N hilos de courier que recorren rutas por las calles (sin órdenes todavía), render fluido con interpolación, señales, duration y apagado ordenado.	DP.3, DP.8, DP.9 (parte grande)	01 (creación y join), 03 (función miembro en hilo, vector<thread> , shared_ptr), 07 (cv con stop)

3.A Courier (primera versión)

```
enum class CourierState { Idle, ToRestaurant, AtRestaurant, Delivering, Broken };

struct CourierSnapshot {
    // copiable, solo valores: seguro de pasar entre hilos
    std::string id; CourierState state;
    int fromNode, toNode; std::int64_t segStartMs, segEndMs;
    std::size_t load; std::vector<int> deliveryNodes;
};

class Courier {
public:
    Courier(std::string id, int startNode, std::shared_ptr<const CityGraph> g,
           std::shared_ptr<const SimClock> clock, std::shared_ptr<EventLog> log /*, ... */);
    void run(); // cuerpo del hilo: std::thread(&Courier::run, courierPtr)
    CourierSnapshot snapshot() const; // lock breve + copia
    void stop();
private:
    bool ride(const std::vector<int>& path); // false si stop (luego: o avería)
    mutable std::mutex m_;
    std::condition_variable cv_;
    /* estado, nodo actual, segmento actual, bolsa..., bool stop_ = false; */
};
```

3.B Recorrer un camino segmento por segmento

```
bool Courier::ride(const std::vector<int>& path) {
    for (std::size_t i = 0; i + 1 < path.size(); ++i) {
        std::unique_lock<std::mutex> lk(m_);
        if (stop_) return false;
        std::int64_t now = clock->nowMs();
        double meters = graph->segmentMeters(path[i], path[i+1]);
        fromNode_ = path[i]; toNode_ = path[i+1];
        segStartMs_ = now; segEndMs_ = now + msToTravel(meters); // speedKmh -> ms
        if (!sleepUntilSim(lk, segEndMs_)) return false; // interrumpible
        currentNode_ = path[i+1];
    }
    return true;
}
```

Mientras duerme en **wait_until**, el mutex está **liberado**, así que el render, los quotes y el dispatcher pueden leer el snapshot sin esperar.

3.C Render (solo en main)

1. Carga la imagen una vez. Si es muy grande, rediménsionala una vez al tamaño de la ventana (por ejemplo, máximo 1280 px de ancho) y escala los píxeles de los nodos con el mismo factor.
2. Precalcula los píxeles de cada nodo.
3. En cada frame: **frame = base.clone()**; dibuja restaurantes (cuadrados), entregas activas (círculos huecos) y cada courier en su posición interpolada, con color por estado.
4. Panel de contadores (**log->summary()**): created, delivered, rejected, pending, breakdowns, avg (min).
5. Leyenda y atribución "© OpenStreetMap contributors" (abajo a la derecha, con fondo blanco para que se lea).
6. **cv::imshow + cv::waitKey(33)**. El **waitKey** marca el ritmo de ~30 fps y además procesa los eventos de la ventana.

```

cv::Point2f Renderer::positionOf(const CourierSnapshot& s, std::int64_t simNow) const {
    if (s.segEndMs <= s.segStartMs) return px(s.fromNode);
    double f = double(simNow - s.segStartMs) / double(s.segEndMs - s.segStartMs);
    f = std::clamp(f, 0.0, 1.0);
    return px(s.fromNode) + (px(s.toNode) - px(s.fromNode)) * static_cast<float>(f);
}

```

3.D main: arranque y apagado

```

int main(int argc, char** argv) {
    // 1. parsear args + loadConfig (errores -> stderr, return != 0)
    // 2. cv::setNumThreads(0); (OpenCV sin hilos internos: R2 y TSAN limpio)
    // 3. std::signal(SIGINT/SIGTERM, onSignal)
    // 4. construir grafo, reloj, log, restaurantes, couriers (shared_ptr)
    // 5. crear hilos: std::vector<std::thread> courierThreads;
    //     for (auto& c : couriers) courierThreads.emplace_back(&Courier::run, c); // shared_ptr por valor
    // 6. crear la ventana y dibujar el primer frame
    // 7. std::cout << "Simulation has started..." << std::endl; // endl = flush obligatorio
    // 8. while (!g_stopRequested && !(durationS > 0 && clock->nowMs() >= durationS*1000)) {
    //     renderer.draw(); if (cv::waitKey(33) == 27) break; }
    // 9. secuencia de apagado (sección 5.9) y resumen
}

```

Trampas de la Fase 3

Pasar `*courier` (el objeto) a `std::thread`: `std::thread` copia sus argumentos, así que el hilo trabajaría sobre una copia (y `Courier` con mutex ni siquiera es copiable). Pasa el `shared_ptr` por valor, que mantiene el objeto vivo mientras el hilo corre.

Dormir con `std::this_thread::sleep_for` durante el viaje: no se puede interrumpir, así que un Ctrl-C durante un segmento largo tarda más de 5 s.

Imprimir `Simulation has started...` antes de crear los hilos, o sin flush: el test empieza a observar demasiado pronto o nunca ve la línea (la salida se bufferiza al redirigirla a un archivo).

Dibujar llamando a `courier->algo()` varias veces por frame: toma una sola `snapshot()` por courier y dibuja desde la copia.

Los puntos "saltan" porque el courier actualiza la posición solo al llegar al nodo. Usa la interpolación.

Probar el apagado solo con ESC: pruébalo con `kill -INT` y con `kill -TERM` desde otra terminal.

Checkpoint Fase 3

Con couriers recorriendo rutas aleatorias por el grafo, los puntos se mueven **suave** sobre las calles y respetan **oneWay**.

`./delivery_sim ../config/equipetrol.json & sleep 5; kill -INT $!; wait $!; echo $?` imprime 0 y termina en menos de 5 s.

Con `durationS: 30` y `timeScale: 60` termina solo a los 0.5 s reales.

Build TSAN: cero reportes.

Cuando pases a la Fase 5, **quita las rutas aleatorias**: un courier sin trabajo debe quedarse quieto (idle).

Preguntas de defensa

- ¿Qué pasa si destruyes un `std::thread` sin `join()`?
- ¿Por qué `wait_until` con predicado y no `sleep_for`?
- ¿Por qué el handler de señal no llama a `stop()` directamente? (Pista: tomar un mutex en un handler no es seguro, porque el handler puede interrumpir a un hilo que ya lo tiene.)

Fase 4: Generador de órdenes y OrderBook

Objetivo	Rúbrica	Conceptos del curso
Órdenes continuas y en ráfagas, reproducibles con la seed; libro acotado; rechazos <code>queueFull</code> y <code>unreachable</code> .	DP.2 (6 pts)	07 (monitor, productor/consumidor acotado), 02 (move), 04/07 (espera con tiempo)

4.A OrderGenerator

```

void OrderGenerator::run() {
    std::mt19937 rng(seed_); // SOLO este hilo lo usa: no hay race
    std::int64_t nextAt = 0; long nextId = 0;
    while (true) {
        nextAt += drawInterval(rng); // exponencial con media meanIntervalMs
        { std::unique_lock<std::mutex> lk(m_);
            if (!sleepUntilSim(lk, nextAt)) return; } // interrumpible por stop()
        int n = drawBurstSize(rng); // 1 normalmente; a veces 2..burstMax
        for (int k = 0; k < n; ++k) {
            Order o = makeOrder("o" + std::to_string(nextId++), rng); // restaurante, entrega, prep
            log_>orderCreated(o.id, o.restaurantId); // SIEMPRE antes de publicarla
            if (!reachable(o)) { log_>orderRejected(o.id, "unreachable"); continue; }
            if (auto back = book_>tryPush(std::move(o))) // devuelve la orden si lleno
                log_>orderRejected(back->id, "queueFull");
        }
    }
}

```

- **Reproducibilidad:** una sola `std::mt19937(seed)` en el generador, y **todas** las decisiones de las órdenes (intervalo, ráfaga, restaurante, entrega y tiempo de preparación) salen de ella en orden fijo. Documenta que la distribución de las órdenes es reproducible y que el resto depende del planificador del sistema operativo.
- `std::exponential_distribution` no da los mismos números en distintas implementaciones de la librería estándar. Si quieres reproducibilidad también entre compiladores, calcula tú `-mean * ln(1-u)` con `u = rng() / 2^32`.
- Ráfagas: por ejemplo, con probabilidad 0.15 el tamaño es uniforme en `[2, burstMax]`; si no, 1. Pueden ser del mismo restaurante o de distintos. Documenta la regla.
- Entrega: un nodo uniforme del grafo, distinto del nodo del restaurante.
- `unreachable: distFromRestaurant[r][delivery] == inf`, o el restaurante no es alcanzable desde `startNode`.

4.B OrderBook (monitor)

```

class OrderBook {
public:
    OrderBook(std::size_t maxPending, std::shared_ptr<const SimClock> clock, std::int64_t acceptTimeoutMs);
    std::optional<Order> tryPush(Order o); // Generator: nullopt = aceptada; si no, te la devuelve
    std::optional<Order> takeForDispatch(); // Dispatcher: bloquea; nullopt SOLO al apagar
    void putBack(Order o); // Dispatcher: nadie la aceptó, esperar capacidad
    void finishDispatch(); // Dispatcher: la orden salió (asignada o rechazada)
    void notifyCapacity(); // Couriers: se liberó espacio en alguna bolsa
    std::vector<int> waitingDeliveryNodes() const; // Render
    void stop();
private:
    mutable std::mutex m_; std::condition_variable cv_;
    std::deque<Order> fresh_; // nunca cotizadas
    std::deque<Order> retry_; // cotizadas sin éxito, esperan capacidad
    std::size_t inDispatch_ = 0; // en manos del dispatcher: cuentan para el cupo
    std::uint64_t capacityGen_ = 0, seenGen_ = 0;
    const std::size_t maxPending_; bool stop_ = false; /* clock, timeout */
};

```

Cupo: `fresh_.size() + retry_.size() + inDispatch_ < maxPending_`. Las órdenes cuentan hasta que se asignan o se rechazan, así que "el libro nunca tiene más de `maxPending` órdenes esperando" se cumple en todo instante.

4.C Lógica de takeForDispatch (el corazón del canal 1)

```

std::unique_lock<std::mutex> lk(m_);
for (;;) {
    if (stop_) return std::nullopt;
    if (!fresh_.empty()) { pop front; ++inDispatch_; return orden; }
    if (!retry_.empty() && capacityGen_ != seenGen_) { // alguien liberó capacidad: reintentar todas
        seenGen_ = capacityGen_; mover retry_ -> fresh_; continue; }
    if (hay en retry_ una orden con created + acceptTimeout <= now) {
        sacarla; ++inDispatch_; return orden; // el dispatcher la rechazará con noCourier
    }
    auto pred = [&]{ return stop_ || !fresh_.empty() || capacityGen_ != seenGen_; };
    if (retry_.empty()) cv_.wait(lk, pred);
    else cv_.wait_until(lk, clock_->toReal(vencimientoMasProximo), pred);
}

```

Por qué el contador de generación: si un courier libera capacidad justo mientras el dispatcher está cotizando (sin el lock), un simple **notify** se perdería, porque nadie estaba esperando todavía. Con el contador, el cambio queda **registrado en el estado** y el predicado lo ve después. Esta es la respuesta a "despertares que no traen noticias" y a los despertares perdidos.

Trampas de la Fase 4

Logear **orderCreated** después de **tryPush**: el dispatcher podría registrar **orderAssigned** antes y el log mostraría una orden asignada que no existe todavía.

Rechazar por **queueFull** usando solo **fresh_.size()** e ignorar las que tiene el dispatcher en la mano.

Usar **notify_one** cuando hay más de un tipo de espera sobre el mismo cv. Aquí solo espera el dispatcher, así que basta con **notify_one**, pero justifícalo.

Dormir con **sleep_for(meanInterval)** en el generador: no es interrumpible.

Checkpoint Fase 4

Dos corridas con la misma seed producen la **misma secuencia** de **orderCreated** (mismo id, restaurante y entrega). Compáralas con **grep orderCreated | cut**.

Con **maxPending: 5** y sin couriers aceptando, se ven rechazos **queueFull** y nunca hay más de 5 esperando.

Con un nodo aislado en la config aparecen rechazos **unreachable**.

Fase 5: Dispatcher simple, restaurantes y ciclo completo

Objetivo	Rúbrica	Conceptos del curso
Ciclo punta a punta: asignar → ir al restaurante → cola FIFO con slots → recoger → entregar. Dispatcher secuencial (una cotización tras otra). noCourier .	DP.5 (8 pts), DP.3 (bolsa)	06 (locks RAII, secciones cortas), 07 (monitor Restaurant, espera por condición)

Guarda esta versión

Haz commit o crea el tag **v-dispatch-secuencial**. El dispatcher secuencial es el **"antes"** del experimento con cambio de diseño que exige DP.10.

5.A Courier::offer (canal 3)

```

// Dispatcher -> Courier. Devuelve nullopt si la aceptó; si no, DEVUELVE la orden (sigue existiendo en un solo lugar).
std::optional<Order> Courier::offer(Order o) {
    std::lock_guard<std::mutex> lk(m_);
    if (stop_ || state_ == CourierState::Broken || load() >= bagCapacity_) return o;
    log->orderAssigned(o.id, id_); // dentro del lock: el courier aún no puede actuar sobre ella
    assigned_.push_back(std::move(o));
    cv_.notify_one(); // despierta al courier si estaba idle
    return std::nullopt;
}
// load() = assigned_.size() + carried_.size() (lo comprometido nunca supera bagCapacity)

```

5.B Bucle del courier

```

void Courier::run() {
    for (;;) {
        Target tgt;
        {
            std::unique_lock<std::mutex> lk(m_);
            state_ = CourierState::Idle;
            cv_.wait(lk, [&]{ return stop_ || !assigned_.empty() || !carried_.empty(); }); // 0% CPU
            if (stop_) return;
            tgt = planNext(); // p.ej. entregar primero lo que llevo; si no, ir al restaurante de assigned_.front()
            state_ = tgt.isPickup ? CourierState::ToRestaurant : CourierState::Delivering;
        } // soltar el lock ANTES de viajar o esperar en otro monitor
        auto path = graph_>shortestPath(currentNode(), tgt.node);
        if (!ride(path)) { if (stopped()) return; handleBreakdown(); continue; } // avería: Fase 7
        if (tgt.isPickup) { if (!pickupAt(*tgt.restaurant)) return; }
        else { if (!deliverAt(tgt.node)) return; }
    }
}

```

5.C Restaurant (monitor con FIFO y slots)

```

class Restaurant {
public:
    // Bloquea hasta que sea el turno de 'courierId' y haya un slot libre. false si stop.
    // Registra pickupStarted de cada orden DENTRO del lock.
    bool acquireSlot(const std::string& courierId, const std::vector<std::string>& orderIds);
    void releaseSlot();
    void stop();
private:
    mutable std::mutex m_; std::condition_variable cv_;
    std::deque<std::string> queue_; int inUse_ = 0; const int slots_; bool stop_ = false;
    std::shared_ptr<EventLog> log_; const std::string id_;
};
// acquireSlot:
// lock; queue_.push_back(me);
// cv_.wait(lk, [&]{ return stop_ || (queue_.front()==me && inUse_ < slots_); });
// if (stop_) { quitar me de queue_; return false; }
// queue_.pop_front(); ++inUse_; for (id : orderIds) log_>pickupStarted(id, me, id_);
// cv_.notify_all(); return true; // el siguiente en la fila revisa si también cabe

```

5.D pickupAt y deliverAt

```

bool Courier::pickupAt(Restaurant& r) {
    auto ids = idsOfAssignedFor(r); // copia bajo m_
    { std::lock_guard<std::mutex> lk(m_); state_ = CourierState::AtRestaurant; }
    if (!r.acquireSlot(id_, ids)) return false; // espera en el cv DEL RESTAURANTE, sin m_ tomado
    bool ok;
    {
        std::unique_lock<std::mutex> lk(m_);
        ok = sleepUntilSim(lk, std::max(clock_>nowMs(), latestReadyAt(ids)) + kHandoverMs);
        if (ok) for (cada orden de r en assigned_) {
            log_>orderPickedUp(o.id, id_, r.id());
            carried_.push_back(std::move(o)); // assigned_ -> carried_ por move
        }
    }
    r.releaseSlot(); // SIEMPRE, incluso si ok == false
    return ok;
}
// deliverAt: sleepUntilSim(now + kDropoffMs); log_>orderDelivered(...); sacar de carried_;
// soltar el lock; fleet_>notifyCapacity(); book_>notifyCapacity();

```

5.E Dispatcher secuencial (versión inicial)

```

void Dispatcher::run() {
    while (auto o = book_>takeForDispatch()) {
        if (clock_>nowMs() - o->createdMs >= acceptTimeoutMs_) {
            log_>orderRejected(o->id, "noCourier"); book_>finishDispatch(); continue; }
        auto ranked = rankCandidatesSequential(*o); // ETA de cada courier, UNA TRAS OTRA
        bool assigned = false;
        for (auto& c : ranked) {
            auto back = c->offer(std::move(*o));
            if (!back) { assigned = true; break; }
            *o = std::move(*back); // la recuperamos y probamos el siguiente
        }
        if (assigned) book_>finishDispatch(); else book_>putBack(std::move(*o));
    }
}

```

Trampas de la Fase 5

Sostener `Courier::m_` mientras esperas en `Restaurant::acquireSlot`: el render se congela y la jerarquía de locks se rompe.

No liberar el slot en el camino de stop, o si hay una excepción. Considera un pequeño guard RAII (`SlotGuard`) cuyo destructor llame `releaseSlot()`.

Loguear `pickupStarted` fuera del lock del restaurante: el orden del log deja de coincidir con el orden de atención y el validador marca violación de FIFO.

Usar `std::move(*o)` y después leer `o->id`: después del move el objeto queda en estado válido pero no especificado.

Sin trabajo, el courier debe quedar **quieto** en `cv_.wait`. Quita las rutas aleatorias de la Fase 3.

Checkpoint Fase 5

Hay órdenes entregadas de punta a punta, y en pantalla se ven puntos azules que se vuelven verdes y luego grises.

Con `pickupSlots: 1` en todos los restaurantes y `burstMax` alto nunca hay dos couriers recogiendo a la vez en el mismo restaurante (lo verifica el validador de la Fase 8; mientras tanto, revisa el log a mano).

Con `bagCapacity: 1`, ningún courier tiene más de una orden.

Con couriers idle y casi sin órdenes, `top -p $(pgrep delivery_sim)` muestra solo el costo de dibujar.

`noCourier` aparece con `couriers: 1` y muchas órdenes.

Preguntas de defensa

- ¿Por qué `offer` devuelve `std::optional<Order>` en vez de `bool`?
- ¿Qué garantiza que el log muestre `orderAssigned` antes que `pickupStarted` para la misma orden?
- Con `notify_all` en el restaurante, ¿qué hacen los que despiertan y no son el `front()`? ¿Por qué eso es correcto?

Fase 6: Cotizaciones concurrentes (pool + promise/future)

Objetivo	Rúbrica	Conceptos del curso
Las estimaciones de los candidatos se calculan al mismo tiempo , fuera del camino crítico, con límite <code>quoteTimeoutMs</code> (tiempo real). Una estimación que falla llega como error y no tumba el programa.	DP.4 (8 pts)	01 (<code>hardware_concurrency</code>), 03 (contenedor de hilos), 04 (promise/future, <code>wait_until</code> , excepción entre hilos)

6.A QuotePool

```

struct Quote { std::string courierId; std::int64_t etaMs; };

class QuotePool {
public:
    explicit QuotePool(std::size_t nWorkers);
    void start(); // crea los hilos (NO en el constructor)
    std::future<Quote> submit(std::function<Quote()> job);
    void stop(); // flag + notify_all
    void join(); // join de cada worker
private:
    struct Task { std::function<Quote()> job; std::promise<Quote> result; };
    void workerLoop();
    std::mutex m_; std::condition_variable cv_;
    std::deque<Task> tasks_; bool stop_ = false;
    std::vector<std::thread> workers_;
};

void QuotePool::workerLoop() {
    for (;;) {
        Task t;
        {
            std::unique_lock<std::mutex> lk(m_);
            cv_.wait(lk, [&]{ return stop_ || !tasks_.empty(); });
            if (stop_) return; // las tareas pendientes se destruyen: broken_promise
            t = std::move(tasks_.front()); tasks_.pop_front();
            try { t.result.set_value(t.job()); } // fuera del lock
            catch (...) { t.result.set_exception(std::current_exception()); } // canal de error
        }
    }
}

```

6.B Tamaño del pool (concepto 01b: decisión según el hardware)

```

unsigned hc = std::thread::hardware_concurrency(); // puede devolver 0 (desconocido)
std::size_t nWorkers = std::clamp<std::size_t>(hc > 1 ? hc - 1 : 1, 1, 16);
// -1: main (render) y los couriers/dispatcher ya compiten por núcleos. Más workers que núcleos
// no aceleran un cálculo CPU-bound; solo agregan cambios de contexto.

```

6.C El dispatcher concurrente

```

auto cands = candidates(*o); // snapshots: no averiados y con capacidad
// Si hay más candidatos que workers, prefiltra los K = nWorkers más cercanos en línea recta,
// para que TODAS las estimaciones pedidas corran realmente al mismo tiempo.
auto deadline = std::chrono::steady_clock::now() + std::chrono::milliseconds

```

- **estimate()**: toma **c.snapshot()**, calcula el tiempo restante del trabajo actual, más Dijkstra desde el nodo actual del courier al restaurante, más **max(prepare restante)**, más la distancia del restaurante a la entrega. Todo dividido por la velocidad.
- **Errores reales**: lanza una excepción si el courier está averiado o si no hay ruta. **Errores inyectados para probar**: variable de entorno **DELIVERY_SIM_QUOTE_FAULT=0.1** (probabilidad de lanzar) y **DELIVERY_SIM_QUOTE_DELAY_MS=50** (espera artificial). No agregues campos al JSON, porque el formato está fijado por los revisores.
- **quoteTimeoutMs** es tiempo **real**; **acceptTimeoutMs** es tiempo **simulado**. No los confundas.
- La lambda captura **shared_ptr por valor (c, g)**: si la cotización llega tarde, el courier y el grafo siguen vivos mientras el worker los use.
- **OrderInfo** es una copia ligera (id, nodos, readyAt), no la **Order**, que es move-only y sigue en manos del dispatcher.

Trampas de la Fase 6

Capturar `this` o referencias locales del dispatcher en la lambda: si el worker la ejecuta tarde, lee memoria que ya no existe (ASan lo detecta).

Llamar `f.get()` sin `try`: una excepción de cotización tumba el hilo del dispatcher y el programa termina con `std::terminate`.

Crear los hilos en el constructor del pool **antes** de inicializar el mutex o la cola: usa `start()`.

Calcular cada cotización con un `std::async` nuevo: vuelve el destructor bloqueante (ver 5.4).

Un `deadline` distinto para cada future: el tiempo total pasaría a ser la suma. Usa **un solo deadline** absoluto.

Checkpoint Fase 6

Con `DELIVERY_SIM_QUOTE_FAULT=0.3`, el programa sigue funcionando y en stderr se ven los errores.

Con `DELIVERY_SIM_QUOTE_DELAY_MS=500` y `quoteTimeoutMs: 200`, las cotizaciones tardías se ignoran y la ventana **no** se congela.

TSAN limpio con 64 couriers y una tasa alta de órdenes.

Preguntas de defensa

- ¿Qué pasa con el future de una cotización que llegó tarde? ¿Y con su promise?
- ¿Por qué no `std::async` aquí, si sí en el precálculo?
- ¿Cómo viaja una excepción del worker al dispatcher?

Fase 7: Averías y handover todo-o-nada

Objetivo	Rúbrica	Conceptos del curso
Avería aleatoria durante un viaje; las órdenes no entregadas pasan atómicamente a otro courier; dos averías simultáneas no bloquean.	DP.6 (8 pts)	06 (dos locks a la vez sin deadlock), 02 (órdenes por move), 07 (espera de capacidad)

7.A Cuándo se avería

Al iniciar cada viaje (hacia el restaurante o hacia el cliente), el courier sortea con su propia `std::mt19937(seed + 1 + índice)` si se averiará, con probabilidad `breakdownProbability`. Si sí, elige un segmento y una fracción del recorrido. En ese punto termina el `wait_until` antes de tiempo (calcula `segEndMs` recortado) y deja su posición congelada ahí.

7.B Transferencia atómica

```
// Todo o nada: se mueven TODAS las órdenes de 'from' a 'to', o NINGUNA.
bool transferBag(Courier& from, Courier& to, EventLog& log) {
    if (&from == &to) return false;
    std::scoped_lock lk(from.m_, to.m_);          // algoritmo de evasión de deadlock (como std::lock)
    if (to.stop_ || to.state_ == CourierState::Broken) return false;
    if (to.load() + from.load() > to.bagCapacity_) return false;
    for (auto& o : from.assigned_) { log.orderReassigned(o.id, from.id_, to.id_);
                                     to.assigned_.push_back(std::move(o)); }
    for (auto& o : from.carried_) { log.orderReassigned(o.id, from.id_, to.id_);
                                    to.carried_.push_back(std::move(o)); }
    from.assigned_.clear(); from.carried_.clear();
    to.cv_.notify_one();
    return true;
} // hazla friend de Courier o un método estático
```

Por qué es atómica: cualquier observador (render, quotes, dispatcher, validador vía log) tiene que tomar `m_` de uno de los dos couriers para ver sus bolsas. Mientras `transferBag` tiene **ambos** locks nadie puede mirar, así que solo se ve el "antes" (todo en A) o el "después" (todo en B).

Por qué no hay deadlock: si A se avería y transfiere a B mientras B se avería y transfiere a A, `scoped_lock(A,B)` y `scoped_lock(B,A)` usan el algoritmo de `std::lock` (intenta, retrocede y reintenta en otro orden), que nunca deja dos hilos esperándose mutuamente. Además, uno de los dos verá al otro `Broken` y devolverá `false`. Lo único que se toma dentro es `EventLog::m_`, que está más abajo en la jerarquía.

7.C handleBreakdown

```
void Courier::handleBreakdown() {
    std::int64_t repairAt;
    { std::lock_guard<std::mutex> lk(m_);
      state_ = CourierState::Broken; // ANTES de intentar transferir
      log_>courierBreakdown(id_);
      repairAt = clock_>nowMs() + kRepairMs; }
    for (;;) {
        if (bagEmpty() || stopped()) break;
        std::uint64_t gen = fleet_>capacityGen();
        for (auto& other : fleet_>byDistanceFrom(currentNode())) // snapshots: sin locks
            if (transferBag(*this, *other, *log_)) break;
        if (bagEmpty()) break;
        // nadie tenía espacio: esperar a que alguien libere, a la reparación o al stop (sin CPU)
        if (!fleet_>waitCapacityChange(gen, clock_>toReal(repairAt))) break;
        if (clock_>nowMs() >= repairAt) break;
    }
    { std::unique_lock<std::mutex> lk(m_); if (!sleepUntilSim(lk, repairAt)) return; }
    { std::lock_guard<std::mutex> lk(m_); state_ = CourierState::Idle; }
    fleet_>notifyCapacity(); book_>notifyCapacity();
}
```

Decisión que debes tomar y documentar	Recomendación
¿La avería es permanente o se repara?	Se repara tras kRepairMs simulados (constante en el código, por ejemplo 10 min). Con breakdownProbability: 1 el sistema sigue avanzando en lugar de quedarse sin couriers.
¿Qué se transfiere: solo lo recogido o también lo asignado?	Ambos (assigned_ y carried_), porque ambos ocupan la bolsa.
¿Si nadie tiene espacio?	Esperar un cambio de capacidad o la reparación (sin CPU). Si se repara antes, conserva sus órdenes. Documentalo.
¿El receptor va a buscar las órdenes físicamente?	Versión simple: la propiedad cambia al instante. Versión realista: las órdenes transferidas llevan un pickupNode = posición del averiado y el receptor pasa por ahí antes. Elige una y explícala.
¿Transferir todo a uno o repartir?	Todo a uno (más simple de razonar). Repartir también es válido si cada orden se mueve atómicamente.

Trampas de la Fase 7

Tomar primero **from.m_** y después, en otra instrucción, **to.m_** con dos **lock_guard**: es un deadlock clásico con dos averías cruzadas. Usa **scoped_lock** con ambos.

Elegir el receptor con el lock tomado y luego intentar tomar el suyo: decide con **snapshots** (sin locks) y **vuelve a verificar** dentro de **scoped_lock**.

Dejar la bolsa vacía y luego "meter" las órdenes al receptor en otra sección crítica: hay un instante en que la orden no está en ninguna bolsa.

Reintentar el handover con **sleep_for(100ms)** en un bucle: es coordinación por sueño (R6). Usa **Fleet::waitCapacityChange**.

Checkpoint Fase 7

Con **breakdownProbability: 1** corre, avanza y termina limpio con Ctrl-C.

El validador confirma que cada **orderReassigned** tiene **from** = courier actual de la orden, y que ninguna orden está en dos bolsas.

TSAN limpio y nunca se cuelga en 20 corridas seguidas con **p = 1**.

Preguntas de defensa

- Explica paso a paso por qué **scoped_lock(a, b)** y **scoped_lock(b, a)** simultáneos no hacen deadlock.
- ¿Qué observador podría ver una orden en ninguna bolsa si no usaras dos locks a la vez?

7. Fase 8: validador del log y escenarios de prueba

Los revisores validan con un programa **cada** regla de las secciones 4, 6 y 7. Tú necesitas el tuyo: `tools/validate_log.py` (Python está permitido como herramienta; no forma parte del programa). Uso: `python3 tools/validate_log.py config.json events.log stdout.txt`.

7.1 Reglas que debe comprobar

#	Regla	Cómo
V1	Cada línea del log es JSON válido y tiene <code>t</code> (int) y <code>event</code> (nombre válido)	<code>json.loads</code> por línea
V2	Campos obligatorios por evento (<code>order</code> , <code>courier</code> , <code>restaurant</code> , <code>from</code> , <code>to</code> , <code>reason</code>)	tabla de la §6.2 del brief
V3	<code>t</code> no decrece para cada orden	último <code>t</code> por orden
V4	Transiciones válidas: <code>created</code> → <code>assigned</code> → <code>pickupStarted</code> → <code>pickedUp</code> → <code>delivered</code> ; <code>rejected</code> solo antes de asignarse	máquina de estados por orden
V5	Máximo un estado final por orden; nada después de un estado final	set de finalizadas
V6	<code>orderReassigned.from</code> es el courier actual de la orden	diccionario orden → courier
V7	La carga por courier (asignadas + llevadas) nunca supera <code>bagCapacity</code>	contador por courier en cada evento
V8	Recogiendo a la vez en un restaurante ≤ <code>pickupSlots</code>	intervalos <code>pickupStarted..pickedUp</code> por courier (no por orden)
V9	Ningún <code>orderCreated</code> después de <code>simulationStopping</code> ; <code>simulationStopping</code> aparece exactamente una vez	posición en el archivo
V10	El resumen es la última línea de stdout y la línea <code>Simulation has started...</code> aparece antes	leer stdout
V11	Contadores del resumen == conteo de líneas del log	comparar uno por uno
V12	<code>created == delivered + rejected + pending</code> ; <code>sum(deliveredPerCourier) == delivered</code>	aritmética
V13	<code>avgDeliveryMs</code> y <code>maxDeliveryMs</code> coinciden con lo que se calcula desde el log	recalcular

7.2 Escenarios (los mismos que usarán los revisores)

Archivo	Qué cambia respecto a equipetrol.json	Qué comprueba
<code>small.json</code>	~6 nodos, 2 couriers, 1 restaurante	Casos borde y arranque
<code>large.json</code>	Cientos de nodos (grilla sintética generada con un script), 64 couriers , <code>meanIntervalMs</code> muy bajo	Rendimiento, fluidez de la ventana, TSAN bajo carga
<code>burst.json</code>	<code>burstMax: 200</code> , <code>maxPending</code> bajo	<code>queueFull</code> , cupo en todo instante
<code>slots1.json</code>	<code>pickupSlots: 1</code> en todos	FIFO y exclusión
<code>bag1.json</code>	<code>bagCapacity: 1</code>	Capacidad
<code>breakdown1.json</code>	<code>breakdownProbability: 1</code>	Handover y que no se cuelgue
<code>unreachable.json</code>	Nodos aislados o solo alcanzables por un <code>oneWay</code> de ida	<code>unreachable</code>
<code>idle.json</code>	Couriers, pero <code>meanIntervalMs</code> enorme	CPU casi 0 (DP.3)
<code>invalid_*.json</code>	Sin <code>fleet</code> ; nodo inexistente; JSON roto; tipo incorrecto; couriers 0; imagen inexistente	Exit ≠ 0 y mensaje en stderr

7.3 Corredor de escenarios

```
#!/usr/bin/env bash
# tools/run_scenarios.sh <binario> <config> <N>
BIN=$1; CFG=$2; N=${3:-20}
for i in $(seq 1 "$N"); do
    LOG=$(mktemp); OUT=$(mktemp)
    "$BIN" "$CFG" --log "$LOG" > "$OUT" &
    PID=$!
    sleep $(( RANDOM % 15 + 1 ))          # (sleep en el shell de pruebas: permitido)
    if (( RANDOM % 2 )); then SIG=INT; else SIG=TERM; fi
    START=$(date +%s.%N); kill -$SIG $PID; wait $PID; CODE=$?; END=$(date +%s.%N)
    echo "run $i sig=$SIG exit=$CODE secs=$(echo "$END - $START" | bc)"
    python3 tools/validate_log.py "$CFG" "$LOG" "$OUT" || echo "  VALIDATION FAILED run $i"
done
```

Prueba también señales **inmediatamente** después de **Simulation has started...** (el brief lo menciona): lee el stdout hasta ver la línea y manda la señal en ese momento.

7.4 Medir el CPU en idle

```
./delivery_sim ../config/tests/idle.json & PID=$!
sleep 10; ps -o %cpu= -p $PID          # o: top -H -p $PID (ver CPU POR HILO)
kill -INT $PID
```

top -H es clave: el hilo main gasta algo por dibujar (está permitido), pero los hilos de courier, dispatcher, workers y generador deben estar en **0.0**.

8. Fase 9: análisis con herramientas (DP.11)

Requisito: **al menos una herramienta de memoria y una de races**, cada una observando una **corrida completa** de `config/equipetrol.json` que incluya ráfagas, averías y apagado por señal. Para que haya averías y ráfagas seguras, usa una copia `analysis/equipetrol_tools.json` con `breakdownProbability` alto (0.2) y `burstMax` alto, y explícalo. Guarda la salida **completa y sin editar** en `analysis/`.

8.1 ThreadSanitizer (races) — obligatorio

```
mkdir -p build-tsan && cd build-tsan && cmake -DENABLE_TSAN=ON .. && make
TSAN_OPTIONS="halt_on_error=0 second_deadlock_stack=1 history_size=4 log_path=./analysis/tsan" \
./delivery_sim ../config/equipetrol.json --log ../analysis/tsan_events.log > ../analysis/tsan_stdout.txt &
PID=$!; sleep 60; kill -INT $PID; wait $PID; echo "exit=$?"
ls ../analysis/tsan.* # si NO existe ningún archivo tsan.<pid>, no hubo reportes
```

- Versión: `gcc --version` (TSAN viene con libtsan de GCC) o `clang --version`.
- Si ves **FATAL: ThreadSanitizer: unexpected memory mapping: sudo sysctl vm.mmap_rnd_bits=28** (kernels nuevos con ASLR alto). Anótalo en el README.
- Reportes **dentro** de libgtk/libglib/OpenCV (código no instrumentado): primero llama a `cv::setNumThreads(0)`. Si quedan, crea `tsan.supp` con `called_from_lib:libgtk-3.so` y similares, y **explica cada línea** en el README. Nunca suprimas un reporte que pase por tu código.

8.2 AddressSanitizer + LeakSanitizer (memoria)

```
mkdir -p build-asan && cd build-asan && cmake -DENABLE_ASAN=ON .. && make
ASAN_OPTIONS="detect_leaks=1:log_path=./analysis/asan" LSAN_OPTIONS="suppressions=../lsan.supp" \
./delivery_sim ../config/equipetrol.json --log ../analysis/asan_events.log > ../analysis/asan_stdout.txt &
PID=$!; sleep 60; kill -TERM $PID; wait $PID; echo "exit=$?"
```

Las fugas típicas de fontconfig/GTK (no son tuyas) van a `lsan.supp` (`leak:libfontconfig`, etc.), cada una **explicada**.

8.3 Valgrind Memcheck (memoria, alternativa o complemento)

```
cd build && valgrind --version
valgrind --leak-check=full --show-leak-kinds=definite,indirect --track-origins=yes \
--log-file=./analysis/memcheck.txt ./delivery_sim ../config/equipetrol.json --log ../analysis/mc_events.log &
PID=$!; sleep 120; kill -INT $PID; wait $PID
```

Valgrind ejecuta unas 20 a 50 veces más lento: sube `timeScale` en la copia de la config. Helgrind/DRD son opcionales. Suelen dar falsos positivos con `std::condition_variable` de libstdc++; si los usas, explícalos.

8.4 Qué va al README por cada herramienta

Campo	Ejemplo
Herramienta y versión	ThreadSanitizer (GCC 13.2.0)
Comando exacto	el bloque de arriba, copiado tal cual
Qué reportó (1ra vez)	"data race en Courier::segEndMs_ leído en Renderer::draw sin lock"
Qué cambiaste	"el render ahora usa Courier::snapshot() que copia bajo m_"
Resultado al re-correr	"0 reportes" + archivo en <code>analysis/</code>
Reportes que quedan	Lista + explicación de por qué no son defectos tuyos (con la supresión usada)

Consejo: introduce **a propósito** un race (por ejemplo, lee `state_` sin lock en el render), córralo con TSAN, guarda esa salida como "antes" y arrégalo. Así tienes un hallazgo real, un fix y un re-run para el informe, y entiendes cómo lee TSAN tu código.

9. Fase 10: experimentos (DP.10)

Reglas: **una sola variable cambia**, todo lo demás igual (misma config y misma seed), resultados reproducibles, tabla comparativa, conclusión que sale de los números y **al menos un experimento que produzca un cambio de diseño**, con números antes y después. Corre cada variante unas 5 veces y reporta media $\pm$ desviación.

9.1 Experimentos sugeridos

#	Varía	Fijo	Mide	Hipótesis
E1 (cambio de diseño)	Cotización secuencial (tag de la Fase 5) vs concurrente (Fase 6), con <code>DELIVERY_SIM_QUOTE_DELAY_MS=20</code>	config, seed, 16 couriers	latencia de asignación (<code>t_assigned - t_created</code>), <code>noCourier</code> , <code>avgDeliveryMs</code>	Secuencial: $\sim 20 \text{ ms} \times$ candidatos, y muchos caen fuera de <code>quoteTimeoutMs</code>
E2	Tamaño del pool: 1, 2, 4, <code>hardware_concurrency()</code>	resto igual	latencia de asignación	Mejora hasta $\sim$ núcleos y luego se estanca (Amdahl)
E3	Nº de couriers: 4, 8, 16, 32	tasa de órdenes	<code>delivered</code> , <code>avgDeliveryMs</code> , <code>rejected</code>	Rendimientos decrecientes
E4	<code>pickupSlots</code> 1 vs 3	resto igual	tiempo en cola del restaurante (<code>pickupStarted - llegada</code>)	Con <code>slots=1</code> el restaurante es el cuello de botella
E5	<code>notify_all</code> vs <code>notify_one</code> , o flush del log por línea vs buffer	resto igual	CPU, throughput	Costo de despertares innecesarios

9.2 Plantilla de tabla

Variante	Runs	created	delivered	rejected	avg assign (ms sim)	avgDeliveryMs	CPU %
Secuencial	5						
Concurrente (P=hc-1)	5						

Guarda en `experiments/E1/` la config, el log y el stdout de **cada** corrida que aparezca en la tabla. Un script `tools/metrics.py` calcula las métricas desde los logs.

10. Fase 11: README final (DP.12)

10.1 Índice del README

1. Descripción corta + captura de pantalla + © **OpenStreetMap contributors** (datos bajo ODbL).
2. Dependencias con versiones (g++, cmake, make, OpenCV, nlohmann/json 3.11.3) y SO usado.
3. Build y ejecución (comandos exactos de la §8 del brief + TSAN/ASan).
4. Cómo se hizo el mapa: fuente, bounds, cómo se extrajeron nodos y calles, nodos de forma.
5. Decisiones de diseño (reparación, qué se transfiere, ráfagas, ETA, K candidatos).
6. **Esquema del programa**: todos los hilos, sobre qué hace loop cada uno y todos los canales (sección 4.3 actualizada al código real).
7. Jerarquía de locks y argumento de ausencia de deadlock.
8. **Argumento escrito de ausencia de data races, por objeto compartido** (10.3).
9. **Mapa de conceptos** (10.2) con **archivo:línea** reales.
10. Experimentos (tabla + conclusiones + cambio de diseño).
11. Análisis con herramientas (tabla 8.4 por herramienta).
12. Limitaciones conocidas.

10.2 Mapa de conceptos: borrador de justificaciones

Reemplaza "dónde" con **archivo:línea** reales. Cada justificación: **por qué ese concepto ahí y qué saldría mal sin él**. En la defensa te pueden preguntar cualquier fila.

Carp	Concepto	Dónde	Por qué / qué fallaría sin él
01	Creación explícita y destino deliberado de cada hilo	main.cpp (creación y secuencia de apagado)	Cada hilo se une en un orden definido tras stop() . Sin join , ~thread llama std::terminate ; con detach , el proceso saldría con hilos tocando objetos ya destruidos.
01	Decisión según el paralelismo del hardware	main.cpp (tamaño de QuotePool)	El pool usa hardware_concurrency()-1 workers: más hilos que núcleos para un cálculo CPU-bound solo agrega cambios de contexto (ver E2).
02	Datos al hilo por valor / referencia / move	std::thread(&Courier::run, c) (shared_ptr por valor); lambdas de quotes; Order por move	Por valor lo que el hilo debe poseer; shared_ptr<const> para lo inmutable compartido; move para lo que debe estar en un solo lugar. Por referencia a un local, el hilo podría leerlo tras destruirse.
02	Órdenes por move, en un solo lugar	Order.h , OrderBook::tryPush , Courier::offer , transferBag	Con copia borrada, el compilador impide duplicar una orden: el invariante "nunca entregada dos veces" se garantiza por tipo.
03	Funciones miembro en hilos; contenedores de hilos	vector<std::thread> de couriers y workers	Cada courier es un objeto con su estado y su hilo corre run() . El vector permite unir todos uniformemente al apagar.
03	Propiedad compartida, sin punteros crudos dueños	shared_ptr<Courier/Restaurante/EventLog/OrderBook>	Varios hilos usan el mismo objeto y el último en soltarlo lo destruye. Una cotización tardía mantiene vivo al courier que evalúa.
04	Tarea que devuelve un future con launch policy	Precálculo std::async(std::launch::async, ...)	Una tarea por restaurante en paralelo. Sin launch::async , la política por defecto podría ser deferred y correr todo en serie dentro de get() .
04	promise/future como canal de un disparo	QuotePool::Task::result	Cada cotización se entrega exactamente una vez al dispatcher. Si nadie la espera ya, llenarla no bloquea a nadie (a diferencia del future de std::async).

Carp.	Concepto	Dónde	Por qué / qué fallaría sin él
04	Espera con límite y excepción entre hilos	<code>Dispatcher:</code> <code>wait_until(deadline);</code> <code>worker: set_exception</code>	El dispatcher nunca espera más de <code>quoteTimeoutMs</code> . Un fallo en un worker llega como excepción en <code>get()</code> en vez de matar el hilo.
05	Argumento escrito de ausencia de races por objeto	README §10.3	Ver la sección siguiente.
05	Tipo seguro de copiar/mover por sus miembros especiales	<code>Order</code> (move-only), <code>CourierSnapshot</code> (copiable, solo valores)	El snapshot no contiene punteros al estado interno, así que el render puede usarlo sin locks. <code>Order</code> no se puede copiar, así que no hay alias.
06	Exclusión mutua solo con locks RAII; secciones cortas	Todos los monitores	El lock se libera en todo camino, incluso con excepciones. Se sueltan antes de viajar, dibujar o calcular rutas para no bloquear a otros.
06	Operación con dos locks a la vez sin deadlock	<code>transferBag</code> con <code>std::scoped_lock</code>	El handover necesita ver y modificar ambas bolsas atómicamente. <code>scoped_lock</code> usa el algoritmo de <code>std::lock</code> , por lo que dos averías cruzadas no se bloquean.
07	Monitores	<code>OrderBook</code> , <code>Restaurant</code> , <code>Courier</code> , <code>EventLog</code> , <code>QuotePool</code> , <code>Fleet</code>	Los datos compartidos solo son accesibles por métodos que toman el lock, así que es imposible olvidarlo desde fuera.
07	Esperas por condición robustas a espurios y al apagado	Todos los <code>cv.wait(lk, pred)</code>	El predicado re-verifica el estado real tras cada despertar, y <code>stop_</code> forma parte de todos los predicados, así que el apagado despierta a todos.

10.3 Argumento de ausencia de data races (plantilla por objeto)

Objeto: `OrderBook`

Datos: `fresh_`, `retry_`, `inDispatch_`, `capacityGen_`, `seenGen_`, `stop_`.
 Todo acceso ocurre dentro de un método público que toma `m_` con `lock_guard/unique_lock`; ningún método devuelve referencias ni punteros a esos datos (se devuelven `Orders` por move o copias). Por lo tanto cada par de accesos está ordenado por el mutex (happens-before).
`maxPending_` es const y se inicializa antes de crear hilos.

Objeto: `CityGraph` / `SimClock`

Se construyen en main antes de crear cualquier hilo y no se modifican después (métodos const, compartido como `shared_ptr<const T>`). La creación del `std::thread` sincroniza con el inicio del hilo, así que no hay escrituras concurrentes: no puede haber data race.

Objeto: `g_stopRequested`

`std::atomic<bool>`: las operaciones atómicas nunca constituyen data race.

Objeto: generador RNG de cada hilo

Cada `std::mt19937` pertenece a un solo hilo (variable local de `run()`); no es compartido.

11. Checklist final antes de entregar

Rúbrica	Verificación
DP.0	Clonar en una carpeta limpia → <code>mkdir build && cd build && cmake .. && make</code> → cero warnings. Igual con <code>-DENABLE_TSAN=ON. grep -rE 'pthread_create omp tbb boost jthread semaphore latch barrier detach' src/</code> sin resultados.
DP.1	La imagen muestra los 3 anillos y Av. San Martín; hay atribución en la ventana y en el README; $\geq 25/40/6$; los nodos caen sobre las calles; configs inválidas salen con $\neq 0$.
DP.2	Misma seed → mismas órdenes; <code>maxPending</code> se respeta; <code>queueFull</code> y <code>unreachable</code> aparecen donde corresponde.
DP.3	Todos los puntos se mueven a la vez, por las calles y respetando <code>oneWay</code> ; bolsa $\leq$ capacidad; <code>top -H</code> en idle: hilos de trabajo en 0%; <code>grep -n sleep_for src/</code> solo en tiempo simulado.
DP.4	Cotizaciones concurrentes; timeout; fallo inyectado sobrevivido; <code>noCourier</code> correcto.
DP.5	El validador pasa V8 (slots) y el orden FIFO en <code>slots1.json</code> × 20 corridas.
DP.6	<code>breakdown1.json</code> × 20 corridas: sin cuelgues; V6 OK.
DP.7	El validador pasa todas las reglas en todos los escenarios × 20; TSAN 0 reportes en todos los escenarios.
DP.8	Ventana fluida con <code>large.json</code> ; colores por estado; entregas activas; contadores; atribución.
DP.9	SIGINT/SIGTERM en momentos aleatorios (incluso justo tras la línea de inicio): exit 0, < 5 s, resumen impreso.
DP.10	Tabla + datos crudos en <code>experiments/</code> + cambio de diseño con antes/después.
DP.11	<code>analysis/</code> con salidas completas; README con versión, comando, hallazgo, fix y re-run por herramienta.
DP.12	El esquema coincide con el código (mismos nombres de clase); el mapa de conceptos tiene <code>archivo:línea</code> reales.
Entrega	<code>config/equipetrol.json</code> + <code>data/equipetrol.png</code> + README + <code>experiments/</code> + <code>analysis/</code> + CMakeLists + src en el repo.

Apéndice A: prompt base para otra IA

Pega esto **al inicio** de cualquier conversación con otra IA sobre el proyecto, y después haz tu pregunta concreta.

Contexto: estoy haciendo un proyecto universitario individual en C++17 (curso de concurrencia). Es una simulación concurrente de delivery con OpenCV (solo para la ventana) y nlohmann/json. Quiero ENTENDER y escribir yo el código; ayúdame con explicaciones, revisión y pistas.

Restricciones Estrictas del profesor (no me propongas nada que las viole):

- Solo C++17 estándar. PROHIBIDO: `std::jthread`, `std::stop_token`, `std::counting_semaphore`, `std::latch`, `std::barrier`, `std::atomic::wait`, corrutinas, cualquier cosa de C++20.
- PROHIBIDO: OpenMP, TBB, Boost, Qt, `pthread_create/pthread_*` directamente, `cv::parallel_for_`, cualquier thread pool o contenedor concurrente de librería.
- Sincronización solo con `std::thread`, `std::mutex`, `std::lock_guard`, `std::unique_lock`, `std::scoped_lock`, `std::condition_variable`, `std::promise/std::future`, `std::async`, `std::atomic`.
- Nunca `m.lock()/m.unlock()` manual. Nunca `detach()`: todo hilo termina con `join()`.
- PROHIBIDO dormir para esperar a otro hilo y PROHIBIDO cualquier bucle que gire revisando una condición. Esperar a otro hilo = `condition_variable::wait` con predicado. `sleep/wait_until` con tiempo SOLO para representar tiempo simulado o el ritmo de frames.
- Toda espera debe poder despertarse por un flag de stop (apagado en < 5 s).
- Sin punteros crudos dueños (usar `shared_ptr/unique_ptr`), sin `new/delete`.
- Compila sin warnings con `-Wall -Wextra`; CMake >= 3.11; build: `cmake .. && make`.
- Debe pasar ThreadSanitizer sin ningún reporte.

Arquitectura: monitores (clase con mutex privado) `OrderBook`, `Restaurant`, `Courier`, `EventLog`, `QuotePool`, `Fleet`; hilos: `main/render`, `OrderGenerator`, `Dispatcher`, `QuoteWorkers`, `Couriers`.

Jerarquía de locks: `Courier` (nivel 1) -> `OrderBook/Restaurant/QuotePool/Fleet` (2) -> `EventLog` (3).

Mi pregunta: ...

Apéndice B: equivalencias C++20 → C++17

Lo que la IA o internet te sugiera (C++20)	Lo que debes usar (C++17)
<code>std::jthread</code>	<code>std::thread</code> + <code>join()</code> explícito en la secuencia de apagado
<code>std::stop_token</code> / <code>request_stop()</code>	<code>bool stop_</code> dentro de cada monitor + <code>stop()</code> que hace <code>notify_all</code>
<code>std::counting_semaphore</code> (slots)	monitor <code>Restaurant</code> con <code>inUse_</code> , deque FIFO y cv
<code>std::latch</code> (esperar que todos arranquen)	No hace falta: imprime la línea de inicio tras crear los hilos. Si la necesitas: contador + cv
<code>std::atomic<bool>::wait/notify</code>	<code>condition_variable</code> con predicado
<code>std::condition_variable_any</code> + <code>stop_token</code>	<code>condition_variable</code> con <code>stop_</code> en el predicado
<code>std::format</code>	<code>nlohmann::json::dump()</code> o <code>std::ostringstream</code>
<code>std::ranges</code>	algoritmos clásicos de <code><algorithm></code>

Apéndice C: problemas típicos en WSL

Síntoma	Solución
La ventana no aparece	<code>echo \$DISPLAY</code> debe mostrar <code>:0</code> . Actualiza WSL (<code>wsl --update</code> en PowerShell) y reinicia (<code>wsl --shutdown</code>).
Could not find OpenCV	<code>sudo apt install libopencv-dev</code> ; verifica con <code>pkg-config --modversion opencv4</code> .
Compilar desde <code>/mnt/c</code> es lento	Clona el repo en <code>~/</code> dentro de Linux.
TSAN: <code>unexpected memory mapping</code>	<code>sudo sysctl vm.mmap_rnd_bits=28</code>
TSAN reporta dentro de GTK/OpenCV	<code>cv::setNumThreads(0)</code> ; si persiste, supresión <code>called_from_lib</code> : explicada en el README.
Valgrind extremadamente lento	Sube <code>timeScale</code> en la config de análisis; reduce el tamaño de la ventana.
Qt/GTK warnings en stderr	No afectan (stderr no se evalúa), pero no deben salir por stdout.

Fin del documento. Siguiendo paso concreto: **Fase 0**. Cuando tengas tu `CMakeLists.txt`, pásalo a revisión.